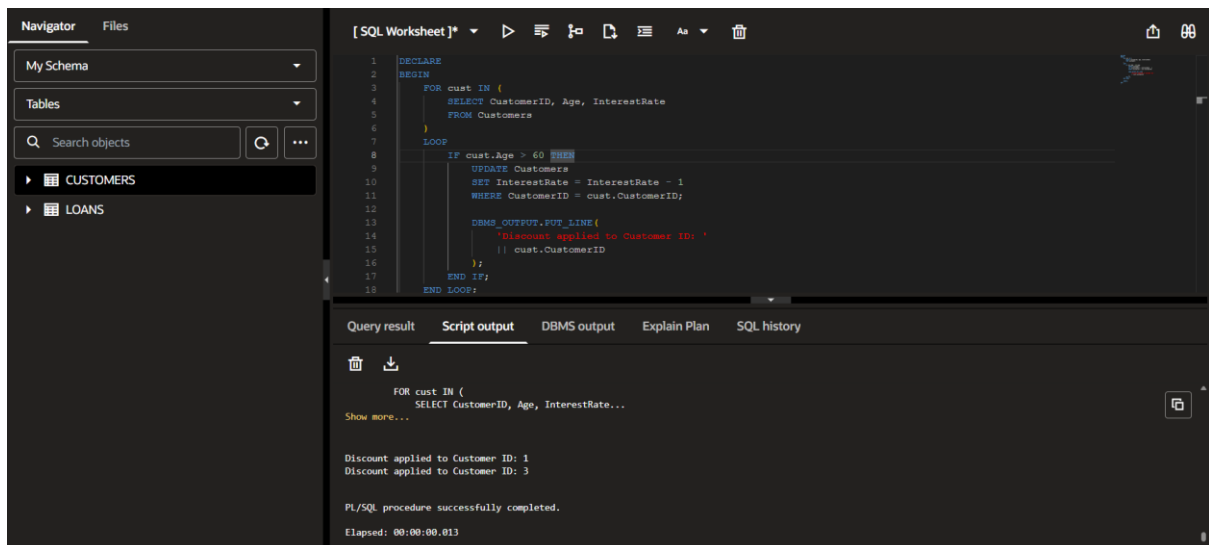


EXCERSISE 1- CONTROL STRUCTURES

SCENARIO 1

The bank wants to apply a discount to loan interest rates for customers above 60 years old.



The screenshot shows an SQL IDE with a dark theme. On the left is a 'Navigator' pane with 'My Schema' and 'Tables' sections. Below 'Tables' is a search bar and a list of objects: 'CUSTOMERS' and 'LOANS'. The main editor area is titled '[SQL Worksheet]*' and contains a PL/SQL script. The script declares a cursor 'cust' to select 'CustomerID', 'Age', and 'InterestRate' from the 'Customers' table. It then enters a loop that checks if 'cust.Age > 60'. If true, it updates the 'InterestRate' by subtracting 1 and outputs a message: 'Discount applied to Customer ID: ' || cust.CustomerID. The script ends with 'END LOOP;' and 'END;'. Below the editor, the 'Script output' pane shows the execution results: 'FOR cust IN (SELECT CustomerID, Age, InterestRate...' followed by 'Discount applied to Customer ID: 1' and 'Discount applied to Customer ID: 3'. It also states 'PL/SQL procedure successfully completed.' and 'Elapsed: 00:00:00.013'.

```
1 DECLARE
2 BEGIN
3   FOR cust IN (
4     SELECT CustomerID, Age, InterestRate
5     FROM Customers
6   )
7   LOOP
8     IF cust.Age > 60 THEN
9       UPDATE Customers
10        SET InterestRate = InterestRate - 1
11        WHERE CustomerID = cust.CustomerID;
12      DBMS_OUTPUT.PUT_LINE(
13        'Discount applied to Customer ID: '
14        || cust.CustomerID
15      );
16    END IF;
17  END LOOP;
18 END;
```

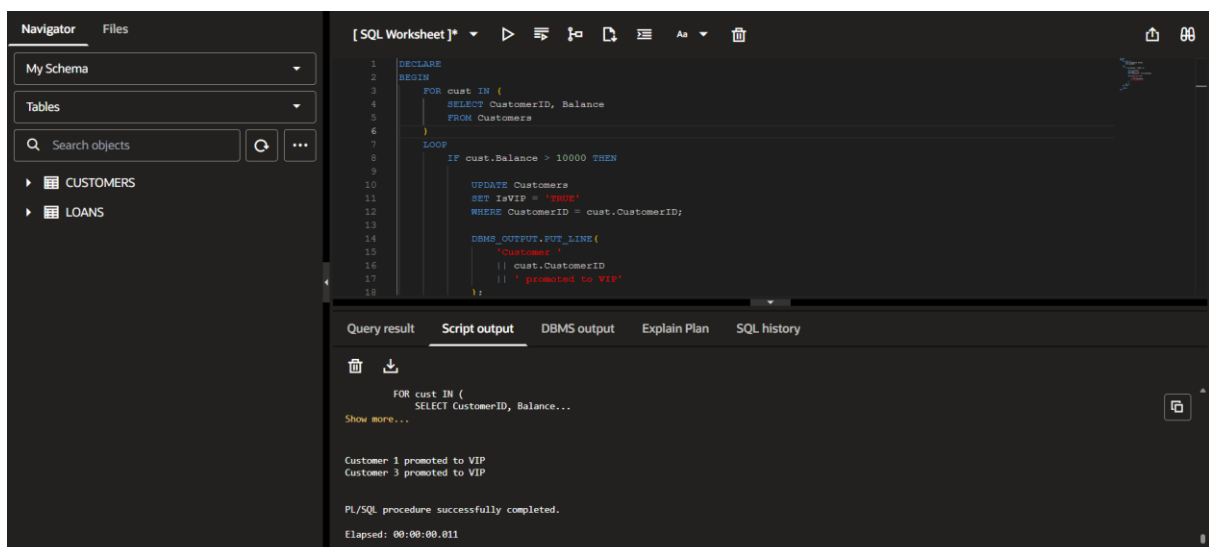
FOR cust IN (SELECT CustomerID, Age, InterestRate...
Show more...

Discount applied to Customer ID: 1
Discount applied to Customer ID: 3

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.013

SCENARIO 2

A customer can be promoted to VIP status based on their balance.



The screenshot shows the same SQL IDE as before. The 'Navigator' pane is identical. The main editor area contains a PL/SQL script that declares a cursor 'cust' to select 'CustomerID' and 'Balance' from the 'Customers' table. It enters a loop that checks if 'cust.Balance > 10000'. If true, it updates the 'IsVIP' column to 'TRUE' and outputs a message: 'Customer ' || cust.CustomerID || ' promoted to VIP'. The script ends with 'END LOOP;' and 'END;'. The 'Script output' pane shows the execution results: 'FOR cust IN (SELECT CustomerID, Balance...' followed by 'Customer 1 promoted to VIP' and 'Customer 3 promoted to VIP'. It also states 'PL/SQL procedure successfully completed.' and 'Elapsed: 00:00:00.011'.

```
1 DECLARE
2 BEGIN
3   FOR cust IN (
4     SELECT CustomerID, Balance
5     FROM Customers
6   )
7   LOOP
8     IF cust.Balance > 10000 THEN
9       UPDATE Customers
10        SET IsVIP = 'TRUE'
11        WHERE CustomerID = cust.CustomerID;
12      DBMS_OUTPUT.PUT_LINE(
13        'Customer '
14        || cust.CustomerID
15        || ' promoted to VIP'
16      );
17    END IF;
18  END LOOP;
19 END;
```

FOR cust IN (SELECT CustomerID, Balance...
Show more...

Customer 1 promoted to VIP
Customer 3 promoted to VIP

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.011

SCENARIO 3

The bank wants to send reminders to customers whose loans are due within the next 30 days.

The screenshot shows the SQL Developer interface. On the left, the Navigator pane displays 'My Schema' with tables 'CUSTOMERS' and 'LOANS'. The main editor shows a PL/SQL procedure named 'SendReminders' that iterates over loans due within the next 30 days and sends reminders via DBMS_OUTPUT. The 'Script output' tab is active, showing the execution results.

```
1 DECLARE
2 BEGIN
3   FOR loan_rec IN (
4     SELECT l.LoanID,
5            c.CustomerID,
6            c.Name,
7            l.DueDate
8     FROM Loans l
9     JOIN Customers c
10    ON l.CustomerID = c.CustomerID
11    WHERE l.DueDate
12          BETWEEN SYSDATE AND SYSDATE + 30
13  )
14  LOOP
15
16    DBMS_OUTPUT.PUT_LINE(
17      'Reminder: Dear ' || c.Name || ', your loan (Loan ID: ' || loan_rec.LoanID || ') is due on ' || l.DueDate || '.
18    )
19  END LOOP;
```

Query result: Script output DBMS output Explain Plan SQL history

FOR loan_rec IN (
 SELECT l.LoanID,...
 Show more...

Reminder: Dear Arun, your loan (Loan ID: 101) is due on 04-JUL-26
Reminder: Dear Rahul, your loan (Loan ID: 103) is due on 14-JUL-26

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.014

EXERCISE 3 – STORED PROCEDURES

SCENARIO 1

The bank needs to process monthly interest for all savings accounts.

BEFORE EXECUTION

The screenshot shows the SQL Developer interface. On the left, the Navigator pane displays 'My Schema' with tables 'ACCOUNTS', 'CUSTOMERS', and 'LOANS'. The main editor shows a stored procedure named 'ProcessMonthlyInterest' that updates the balance of savings accounts by 0.01%. The 'Query result' tab is active, showing the execution results.

```
1 CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
2 IS
3 BEGIN
4   UPDATE Accounts
5   SET Balance = Balance * (Balance * 0.01)
6   WHERE AccountType = 'Savings';
7
8   COMMIT;
9 END;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.019 seconds

	ACCOUNTID	ACCOUNTTYPE	BALANCE
1	101	Savings	10000
2	102	Savings	20000
3	103	Current	15000

AFTER EXECUTION

The screenshot shows the SQL Developer interface. On the left, the Navigator pane displays 'My Schema' with a 'Tables' section containing 'ACCOUNTS', 'CUSTOMERS', and 'LOANS'. The main editor shows a SQL query: `SELECT * FROM Accounts;`. Below the editor, the 'Query result' tab is active, showing the execution time as 0.001 seconds. The result is a table with three columns: ACCOUNTID, ACCOUNTTYPE, and BALANCE.

	ACCOUNTID	ACCOUNTTYPE	BALANCE
1	101	Savings	10100
2	102	Savings	20200
3	103	Current	15000

SCENARIO 2

The bank wants to implement a bonus scheme for employees based on their performance.

The screenshot shows the SQL Developer interface. The main editor contains a PL/SQL procedure: `CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(p_department IN VARCHAR2, p_bonus IN NUMBER) IS BEGIN UPDATE Employees SET Salary = Salary + (Salary * p_bonus / 100) WHERE Department = p_department; COMMIT; END;`. Below the editor, the 'Query result' tab is active, showing the execution time as 0.017 seconds. The result is a table with four columns: EMPLOYEEID, EMPLOYEEENAME, DEPARTMENT, and SALARY.

	EMPLOYEEID	EMPLOYEEENAME	DEPARTMENT	SALARY
1	1	Arun	IT	55000
2	2	Rahul	IT	66000
3	3	Kumar	HR	45000

SCENARIO 3

Customers should be able to transfer funds between their accounts.

BEFORE EXECUTION

NavigatorFiles

My Schema

Tables

Search objects

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

[SQL Worksheet]*

1

```
SELECT * FROM Accounts;
```

Query resultScript outputDBMS outputExplain PlanSQL history

DownloadExecution time: 0.001 seconds

	ACCOUNTID	ACCOUNTTYPE	BALANCE
1	101	Savings	10000
2	102	Savings	5000

AFTER EXECUTION

NavigatorFiles

My Schema

Tables

Search objects

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

[SQL Worksheet]*

1

```
CREATE OR REPLACE PROCEDURE TransferFunds(  
  p_fromAccount IN NUMBER,  
  p_toAccount IN NUMBER,  
  p_amount IN NUMBER  
)  
IS  
  v_balance NUMBER;  
BEGIN  
  SELECT Balance  
  INTO v_balance  
  FROM Accounts  
  WHERE AccountID = p_fromAccount;  
  IF v_balance >= p_amount THEN  
    UPDATE Accounts  
    SET Balance = Balance - p_amount
```

Query resultScript outputDBMS outputExplain PlanSQL history

DownloadExecution time: 0.002 seconds

	ACCOUNTID	ACCOUNTTYPE	BALANCE
1	101	Savings	5000
2	102	Savings	10000